

Reflection of Templates

Document #: P3420R1 [\[Latest\]](#) [\[Status\]](#)
Date: 2025-01-13
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>
Andrei Alexandrescu, NVIDIA
<andrei@nvidia.com>
Daveed Vandevoorde, EDG
<daveed@edg.com>
Michael Garland, NVIDIA
<mgarland@nvidia.com>

Contents

[1 Revision History](#)

[2 Motivation](#)

[2.1 Example: Logging and Forwarding](#)

[2.2 Example: logging_vector](#)

[2.2.1 Renaming and Projection](#)

[3 Metafunctions for Template Declarations](#)

[3.1 Synopsis](#)

[3.1.1 declaration_of](#)

[3.1.2 set_name](#)

[3.1.3 template_alternatives_of](#)

[3.1.4 template_parameters_of](#)

[3.1.5 template_parameter_list](#)

[3.1.6 attributes_of, add_attribute, remove_attribute](#)

[3.1.7 set_requires_clause, add_requires_clause_conjunction, add_requires_clause_disjunction](#)

[3.1.8 is_primary_template](#)

[3.1.9 overloads_of](#)

[3.1.10 is_inline](#)

[3.1.11 is_const, is_explicit, is_volatile, is_rvalue_reference_qualified, is_lvalue_reference_qualified, is_static_member, has_static_linkage, is_noexcept](#)

[3.1.12 set_noexcept_clause, add_noexcept_clause_conjunction, add_noexcept_clause_disjunction](#)

[3.1.13 set_explicit_clause, add_explicit_clause_conjunction, add_explicit_clause_disjunction](#)

3.1.14	is_declared_constexpr, is_declared_consteval
3.1.15	parameters_of
3.1.16	parameter_list
3.1.17	forward_parameter_list
3.1.18	set_trailing_requires_clause, add_trailing_requires_clause_conjunction, add_trailing_requires_clause_disjunction
4	Metafunctions for Iterating Members of Class Templates
5	Future Work
6	References

§ 1 Revision History

Since [\[P3420R0\]](#):

- replaced API that returns portions of a function declaration as a token sequence with a functional-style API that takes a declaration and returns a modified declaration
- for implementation friendliness, removed metafunctions returning code that potentially contains a mix of dependent and non-dependent identifiers as token sequences
- consequently, no need for the “as-if” rule for compiler-returned token sequences

§ 2 Motivation

A key trait that makes a reflection facility powerful is *completeness*—the ability to reflect the entire source language. Current proposals facilitate reflection of certain declarations in a namespace or a [struct/class/union](#) definition. Although [\[P2996R7\]](#)’s `members_of` metafunction includes template members (function template and class template declarations), it does not offer primitives for reflection of template declarations themselves. In this proposal, we aim to define a comprehensive API for reflection of C++ templates.

A powerful archetypal motivator—argued in [\[P3157R1\]](#) and at length in the CppCon 2024 talk “Reflection Is Not Contemplation”—is the *identity* metafunction. This function, given a class type, creates a replica of it, crucially providing the ability to make minute changes to the copy, such as changing its name, adding or removing members, changing the signature of existing member functions, and so on. By means of example:

```
class Widget {  
    int a;  
public:  
    template <class T> requires (std::is_convertible_v<T, int>)  
    Widget(const T&);  
    const Widget& f(const Widget&) const &;  
    template <class T>  
    void g();  
};
```

```

consteval {
    identity(^^Widget, "Gadget");
}
// Same effect as this handwritten code:
// class Gadget {
//     int a;
// public:
//     template <class T> requires (std::is_convertible_v<T, int>)
//     Gadget(const T&);
//     const Gadget& f(const Gadget&) const &;
//     template <class T>
//     void g();
// };

```

While an exact copy of a type is not particularly interesting—and although a compiler could easily provide a primitive to do so, such a feature would miss the point—the ability to deconstruct a type into its components (bases, data members, member functions, nested types, nested template declarations, etc.) and reassemble them in a different context enables a vast array of applications. These include various forms of instrumentation, creating parallel hierarchies as required by several design patterns, generating arbitrary subsets of an interface (the *powerset* of an interface), logging, tracing, debugging, timing measurements, and many more. We consider *identity*'s ability to perform the roundtrip from a type to its components and back to the type the quintessential goal of reflection, which is at the same time the proof of reflection's completeness and the fountainhead of many of its applications.

In order to reflect on nontrivial C++ code and splice it back with controlled modifications, the language must necessarily reflect on all templates—class templates and specializations thereof, function templates, variable templates, and alias templates. This is not an easy task; templates pose unique challenges to a reflection engine because, by their nature, they are only partially analyzed semantically. For example, neither parameter types nor the return type of a function template can be represented as reflections of types because dependent types are not known until the template is instantiated. The same thinking goes for a variety of declarations that can be found inside a class template; C++ templates are more akin to patterns by which code is to be generated than to standalone, semantically verified code. For that reason, the reflection facilities proposed in P2996 intended for non-templated declarations are not readily applicable to templates.

Conversely, reflection code would be seriously hamstrung if it lacked the ability to reflect on template code (and subsequently manipulate and generate related code), especially because templates are ubiquitous in today's C++ codebases. Furthermore, limiting reflection to instantiations of class templates (which would fit the charter of P2996) does not suffice because class templates commonly define inner function templates (e.g., every instantiation of `std::vector` defines several constructor templates and other function templates such as `insert` and `emplace_back`). Therefore, to the extent reflection of C++ declarations is deemed useful, reflection of templates is essential.

Given that many elements of a template cannot be semantically analyzed early (at template definition time), this proposal adopts a two-pronged strategy for reflecting components of template declarations:

- template parameter names and function parameter names can be reflected as *token sequences*, as proposed in [\[P3294R2\]](#);
- the other elements of a template declaration—constraints, predicate in the `noexcept` or `explicit` specifiers, default template arguments, and default function arguments—are kept together with the template declaration;
- new declarations are obtained from existing declarations in a functional manner, and the “deltas” are expressed as token sequences.

The initial proposal [\[P3420R0\]](#) allowed accessing all parts of a template definition as token sequences. However, for some implementations performing such extrication was deemed difficult. Also, separating parts of a function declaration made it difficult to define how names are looked up after splicing.

We paid special attention to preserve the spirit and design style of [\[P2996R7\]](#) and to ensure seamless interoperation with it.

§ 2.1 Example: Logging and Forwarding

As a conceptual stylized example, consider the matter of creating a function template `logged::func` for any free function template `func` in such a way that `logged::func` has the same signature and semantics as `func`, with the added behavior that it logs the call and its parameters prior to execution. We illustrate the matter for function templates because it is more difficult (and more complete) than the case of simple functions.

```
struct Widget {};
```

```
template <typename T>
requires std::is_copy_constructible_v<T>
void fun(const T& value, S&) { ... }
```

```
template <typename... Ts>
void logger(const Ts&.. values);
```

```
// Introspection will add declarations here.
namespace logged {
    struct Widget {}; // for exposition purposes
}
```

```
// Given a function func, create a function logged::func that logs
// arguments.
constexpr void make_logged(std::meta::info f) {
    using namespace std::meta;
    namespace_inject(^^::logged, ^^{
        [:\(declaration_of(f)):] {
            logger("Calling " + identifier_of(f) + "(",
                \tokens(parameter_list(f)), ")");
            return [:\(f):]<\tokens(template_parameter_list(f))>
                (\tokens(forward_parameter_list(f)));
        }
    });
}
```

```

consteval {
    make_logged(^^fun);
}

// Equivalent hand-written code:
// namespace logged {
//     template <typename T>
//     requires std::is_copy_constructible_v<T>
//     void fun(const T& value, const ::Widget& __p0) {
//         logger("Call to ", "fun", "(", value, __p0, ")");
//         return fun<T>(std::forward<T>(value), std::forward<T>(__p0));
//     }
// }

```

The code uses `std::meta::namespace_inject` as defined in [\[P3294R1\]](#) to create a function definition from a token sequence in a given namespace, in this case `logged`. We identify a few high-level metafunctions that facilitate such manipulation of template declarations. First, `std::meta::declaration_of(std::meta::info f)` returns a reflection handle (i.e. an `std::meta::info` value) corresponding to the declaration (prototype) of the function template reflected by `f`. The template parameters, `requires` and `noexcept` clauses (if present), function parameters, and return type are all part of the returned reflection. Most importantly, the declaration can be modified and spliced back into code.

Names used in `declaration_of` are handled in a hygienic manner that prevents unintended binding. All non-dependent names are looked up in the original declaration. Thus, even if namespace `logged` defines a type called `Widget`, the declaration returned by `declaration_of` uses the same `Widget` as the original declaration (which is why we used `::Widget` in the comment representing equivalent code). Dependent names are, as expected, not looked up. The name of the function itself also information about what scope `f` was defined in (in our example the global namespace), but the primitive `namespace_inject` ignores that information and injects a new function of the same name in the given namespace.

The metafunction `template_parameter_list(f)` returns, for the reflection `f` of a function template, a token sequence consisting of the comma-separated list of that function's template parameters. That token sequence can be spliced in code that generates the instantiation of `[ :f: ]` or a template with similar parameters. The names of the parameters are implementation-defined and bear no relationship with the user-given parameter names in the declaration of `f`; this avoids possible ambiguities created by duplicate declarations and also sidesteps awkward issues such as handling unnamed parameters. User code may assume each parameter name is world-unique, which implies that `template_parameter_list(f)` is meaningful only from within a declaration created with `declaration_of` (or from within the function reflected by `f` itself).

Next, `parameter_list(f)` produces a token sequence consisting of the comma-separated list of the parameters of `f`, with pack expansion suffix where applicable. The list can be spliced in code that passes the function parameters to another function call. In our case, `parameter_list` is used to pass all parameters to template function `logger`. Just like with `template_parameter_list`, the parameter names are chosen by the implementation and can be considered world-unique.

Finally, `forward_parameter_list(f)` creates the tokens of a call to the function reflected by `f` with the appropriate insertion of calls to `std::forward`, again with pack expansion suffix where appropriate. The example uses `forward_parameter_list` to pass the generated function's arguments down to the implementation.

To recap, we propose generating new functions (or function templates) from existing ones by splicing `declaration_of` for copying the function prototype, followed by the new function body as a token sequence. To access and pass around template parameters (when appropriate) and function parameters, we propose the helper metafunctions `template_parameter_list`, `parameter_list`, and `forward_parameter_list`. These primitives allow manipulating parameters without needing to name them explicitly.

§ 2.2 Example: logging_vector

Consider a more elaborate example of a class template `logging_vector`, which defines a functional equivalent of `std::vector` but that uses a function logger to log all calls to its member functions. Here, we need to iterate the public members of `std::vector` and insert appropriate copies of declarations.

```
template <typename T, typename A = std::allocator<T>>
class logging_vector {
private:
    std::vector<T, A> data;
    using namespace std::meta;
public:
    consteval {
        template for (constexpr auto r : get_public_members(^std::vector<T,
            A>)) {
            if (is_type_alias(r) || is_class_type(r)) {
                queue_injection(^{ using \id(identifier_of(r)) = [:\(r):]; });
            } else if (is_function(r) && !is_special_member_function(r)) {
                queue_injection(^{
                    [:\(declaration_of(r)):] {
                        logger("Calling ", identifier_of(^std::vector<T, A>), ":",
                            identifier_of(r), "(", \tokens(parameter_list(r)), ")");
                        return data.[:r:](forward_param_list_of(r));
                    }
                });
            } else if (is_function_template(r) && !is_constructor(r)) {
                queue_injection(^{
                    [:\(declaration_of(r)):] {
                        logger("Calling ", identifier_of(^std::vector<T, A>), ":",
                            identifier_of(r), "(", \tokens(parameter_list(r)), ")");
                        return data.template [:\(r):]
                            <\tokens(template_parameter_list(r))>(
                                \tokens(forward_parameters_list(r))
                            );
                    }
                });
            } else if (is_constructor(r)) {
                queue_injection(^{
```

```

                                [:\(declaration_of(r)):]      :
    data(\tokens(forward_parameters_list(r))) {}
    });
} else {
    // Ignore other nested declarations.
}
}
}
};

```

(Refer to [P2996R7] for metafunctions `get_public_members`, `is_type_alias`, `is_class_type`, `is_function`, `is_function_template`, and `identifier_of`, of which semantics should be intuitive as we discuss the context.) Here, as the code iterates declarations in class `std::vector<T, A>` (an instance, not the template class itself, which simplifies a few aspects of the example), it takes action depending on the nature of the declaration found. For `typedef` (or equivalent `using`) declarations and for nested class declaration, a `using` declaration of the form `using name = std::vector<T, A>::name;` is issued for each corresponding `name` found in `std::vector<T, A>`.

If the iterated declaration introduces a non-special function (filtered with the test `is_function(r) && !is_special_member_function(r)`), a new function is defined with the same signature. The definition issues a call to `logger` passing the parameters list, followed by a forwarding action to the original function for the data member.

If the iterated declaration is that of a non-special function template (filtered by testing `is_function_template(r) && !is_constructor(r)`), the code synthesizes a function template. Although the template declaration has considerably more elements, the same `declaration_of` metafunction primitive is used. In the synthesized declaration, all names are already looked up in the context of the `std::vector<T, A>` instantiation; for example, `begin()` returns type `std::vector<T, A>::iterator` and not `logging_vector<T, A>::iterator` (even if that name has been declared as an alias).

Finally, if the iterated declaration is a constructor, it is handled separately because the generated code has a distinct syntax. For constructors (whether templated or not), the generated code simply forwards all parameters to the constructor of the payload.

For seeing how variadic parameters are handled, consider the function `emplace` that is declared as a variadic function template with the signature `template <typename... A> iterator emplace(const_iterator, A&&...);`. For the reflection of `emplace`, calling `template_parameter_list` returns the token sequence `^^{ _T0... }` (exact name chosen by the implementation), which is suitable for passing to the instantiation of another template. Correspondingly, `parameter_list` returns the token sequence `^^{ __p0, __p1... }`. This token sequence can be expanded anywhere the arguments of the called function are needed.

It is worth noting that the forwarding call syntaxes `data.[:r:](...)` (for regular functions) and `data.template [[:r:]<...>(...)` (for function templates) are already defined with the expected semantics in [P2996R7], and work properly in the prototype implementations. The only added elements are the metafunctions `template_parameter_list`, `parameter_list`, and `forward_parameter_list`,

which ensure proper passing down of parameter names from the synthesized function to to the corresponding member function.

§ 2.2.1 Renaming and Projection

The `logging_vector` example has an issue related to member functions of `std::vector` that refer to `std::vector` itself in their signature, such as `void swap(std::vector<T, A>&)`. The example as written above will generate code equivalent to the following:

```
template <typename T, typename A>
class logging_vector {
    ...
    void swap(std::vector<T, A>& other) {
        logger("Calling ", "std::vector<T, A>", ":",
            "swap", "(", other, ")");
        return data.swap(std::forward<decltype(other)>(other));
    }
    ...
};
```

The declaration is technically correct, but not what was intended. We need to define `swap` as taking another instance of `logging_vector`, i.e., `void swap(logging_vector<T, A>&)`:

```
void swap(logging_vector<T, A>& other) {
    logger("Calling ", "std::vector<T, A>", ":",
        "swap", "(", other.data, ")");
    return data.swap(std::forward<decltype(other)>(other).data);
}
```

There are two fundamental transformation we need to perform to morph the original `vector<T, A>::swap` into its desired counterpart inside `logging_vector`:

- *Replacement*: for all parameters in the signature that have type `std::vector<T, A>`, change their type to `logging_vector<T, A>`; and
- *Projection*: for all parameters `p` in the function body that have type `std::vector<T, A>`, replace `p` with `p.data`.

For replacement, we propose a primitive metafunction `replace` that replaces a type with another throughout a declaration, potentially in multiple places, in a manner reminiscent to the [alpha renaming](#) notion found in lambda calculus. To that end we define `replace` for declarations, with the signature:

```
info replace(info declaration, info replace_this, info with_this);
```

The metafunction returns a new declaration with the replacements effected. To improve the `logging_vector` example above, we'd need to replace the uses of `declaration_of(f)` with

`replace(declaration_of(f), ^^std::vector<T, A>, ^^logging_vector<T, A>))`. The construct would transform properly all declarations.

Projection is trickier because not all parameters should be affected, only those whose type (after removing cvref qualifiers) is `std::vector<T, A>`. To effect projection, user code defines a *projection function*:

```
// Inside logging_vector's definition
template <typename X>
decltype(auto) project(X&& x) {
    if constexpr (std::is_same_v<remove_cvref_t<X>,
std::logging_vector>) {
        return std::forward<decltype(x.data)>(x.data);
    } else {
        return std::forward<X>(x);
    }
}
```

We then propose a standard metafunction that applies a projection function to each parameter of a function:

```
info project_parameters(info declaration, info projection_function);
```

Armed with these artifacts, we can have reflection generate proper forwarding functions from existing member functions of `std::vector` like this:

```
// For member functions of std::vector<T, A>
auto my_decl = replace(declaration_of(r), ^^std::vector<T, A>,
^^logging_vector<T, A>);
auto my_params = project_parameters(r, ^^project);
queue_injection(^^{
    [:(decl):] {
        logger("Calling ", identifier_of(^^std::vector<T, A>), "::",
            identifier_of(r), "(", \tokens(my_params), ")");
        return data.[:r:](forward_param_list_of(my_params));
    }
});
```

The code for member function templates is similar. One notable detail is that `forward_param_list_of` works seamlessly on function declarations and on projected parameter lists (flexibility made possible by the uniform representation of code artifacts as `std::meta::info` objects).

§ 3 Metafunctions for Template Declarations

All template declarations (whether a class template or specialization thereof, a function template, an alias template, or a variable template), have some common elements: a template parameter list, an optional list

of attributes, and an optional template-level `requires` clause.

If our goal were simply to splice a template declaration back in its entirety—possibly with a different name and/or with normalized parameter names—the introspection metafunction `declaration_of(f)` may seem sufficient. However, most often code generation needs to tweak elements of the declaration—for example, adding a conjunction to the `requires` clause or eliminating the deprecated attribute. Therefore, we aim to identify structural components of the declaration and define primitives that manipulate each in turn. That way, we offer unbounded customization opportunities by allowing users to amend the original declaration with custom code.

In addition to template parameters, `requires` clause, and attributes, certain template declarations have additional elements as follows:

- Class templates and variable templates may add explicit specializations and partial specializations
- Function templates may create overload sets (possibly alongside regular functions of the same name)
- Alias templates contain a declarator
- Member variables of class templates, and also variable templates, have a type and an optional initializer
- Function template declarations may contain these additional elements:
 - `inline` specifier
 - `static` linkage
 - `noexcept` clause (possibly predicated)
 - cvref qualifiers
 - `explicit` specifier (possibly predicated)
 - `constexpr` or `consteval` specifier
 - return type
 - function parameters
 - trailing `requires` clause
 - function template body

Armed with a these insights and with the strategy of using token sequences throughout, we propose the following reflection primitives for templates.

§ 3.1 Synopsis

The declarations below summarize the metafunctions proposed, with full explanations for each following. Some (where noted) have identical declarations with metafunctions proposed in P2996, to which we propose extended semantics.

```

namespace std::meta {
    // Returns the reflection of the declaration of a template
    consteval auto declaration_of(info) -> info;
    // Replacement
    consteval auto replace(info declaration, info replace_this, info
        with_this) -> info;
    // Projection
    consteval auto project_parameters(info declaration, info
        projection_function) -> info;
    // Returns given declaration with a changed name
    consteval auto set_name(info) -> info;
    // Multiple explicit specializations and/or partial specializations
    consteval auto template_alternatives_of(info) -> vector<info>;
    // Template parameters vector
    consteval auto template_parameters_of(info) -> vector<info>;
    // Comma-separated template parameter list
    consteval auto template_parameter_list(info) -> info;
    // Attributes - extension of semantics in P3385
    consteval auto attributes_of(info) -> vector<info>;
    consteval auto add_attribute(info) -> info;
    consteval auto remove_attribute(info) -> info;
    // Template-level requires clause
    consteval auto set_requires_clause(info, info) -> info;
    consteval auto add_requires_clause_conjunction(info, info) -> info;
    consteval auto add_requires_clause_disjunction(info, info) -> info;
    // Is this the reflection of the primary template declaration?
    consteval auto is_primary_template(info) -> bool;
    // Overloads of a given function name
    consteval auto overloads_of(info) -> vector<info>;
    // Inline specifier present?
    consteval auto is_inline(info) -> bool;
    // cvref qualifiers and others - extensions of semantics in P2996
    consteval auto is_const(info) -> bool;
    consteval auto is_explicit(info) -> bool;
    consteval auto is_volatile(info) -> bool;
    consteval auto is_rvalue_reference_qualified(info) -> bool;
    consteval auto is_lvalue_reference_qualified(info) -> bool;
    consteval auto is_static_member(info) -> bool;
    consteval auto has_static_linkage(info) -> bool;
    consteval auto is_noexcept(info) -> bool;
    // predicate for `noexcept`
    consteval auto set_noexcept_clause(info, info) -> info;
    consteval auto add_noexcept_clause_conjunction(info, info) -> info;
    consteval auto add_noexcept_clause_disjunction(info, info) -> info;
    // `explicit` present? - extension of P2996
    // predicate for `explicit`
    consteval auto set_explicit_clause(info, info) -> info;
    consteval auto add_explicit_clause_conjunction(info, info) -> info;
    consteval auto add_explicit_clause_disjunction(info, info) -> info;
    // `constexpr` present?
    consteval auto is_declared_constexpr(info) -> bool;
    // `consteval` present?
    consteval auto is_declared_consteval(info) -> bool;
    // Function template parameters
    consteval auto parameters_of(info) -> vector<info>;

```

```

consteval auto parameter_list(info) -> vector<info>;
consteval auto forward_parameter_list(info) -> vector<info>;
//
consteval auto set_trailing_requires_clause(info, info) -> info;
consteval auto add_trailing_requires_clause_conjunction(info, info)
    -> info;
consteval auto add_trailing_requires_clause_disjunction(info, info)
    -> info;
}

```

§ 3.1.1 declaration_of

```

consteval auto declaration_of(info) -> info;

```

Returns the reflection of the declaration of a type or template. Subsequently that reflection can be inserted as part of a token sequence. Typical uses issue a call `declaration_of` and then compute a slightly modified declaration before splicing it.

§ 3.1.2 set_name

```

info set_name(info, string_view new_name);

```

Given a reflection, returns a reflection that is identical except for the name which is `new_name`. Example:

```

template <typename T>
void fun(const T& value) { ... }

consteval {
    auto f = ^^fun;
    queue_injection(^^{
        [:\(set_name(declaration_of(f), "my_fun")):]{
            return [:\(f):]<\tokens(template_parameters_of(f))>
                (\tokens(forward_parameter_list(f)));
        }
    }));
}

// Equivalent code:
// template <typename T>
// void my_fun(const T& value) {
//     return fun<T>(value);
// }

```

§ 3.1.3 template_alternatives_of

```

vector<info> template_alternatives_of(info);

```

In addition to general template declarations, a class template or a variable template may declare explicit specializations and/or partial specializations. Example:

```
// Primary template declaration
template <typename T, template<typename> typename A, auto x> class C;
// Explicit specialization
template <> class C<int, std::allocator, 42> {};
// Partial specialization
template <typename T> class C<T, std::allocator, 100> {};
```

Each specialization must be accessible for reflection in separation from the others; in particular, there should be a mechanism to iterate `^C` in the example above to reveal info handles for the three available variants (the primary declaration, the explicit specialization, and the partial specialization). Each specialization may contain the same syntactic elements as the primary template declaration. In addition to those, explicit specializations and partial specializations also contain the specialization's template arguments (in the code above: `<int, std::allocator, 42>` and `<T, std::allocator, 100>`, respectively), which we also set out to be accessible through reflection.

Given the reflection of a class template or variable template, `template_alternatives_of` returns the reflections of all explicit specializations and all partial specializations thereof (including the primary declaration). Declarations are returned in syntactic order. This is important because a specialization may depend on a previous one, as shown below:

```
// Primary declaration
template <class T> struct A { ... };
// Explicit specialization
template <> struct A<char> { ... };
// Explicit specialization, depends on the previous one
template <> struct A<int> : A<char> { ... };
// Partial specialization, may depend on both previous ones
template <class T, class U> struct A<std::tuple<T, U>> : A<T>, A<U> {
    ... };
```

As a consequence of the source-level ordering, the primary declaration's reflection is always the first element of the returned vector. Example:

```
template <typename T> class C;
template <> class C<int> {};
template <class T> class C<std::pair<int, T>> {};
```



```
// Get reflections for the primary and the two specializations
constexpr auto r = template_alternatives_of(^C);
static_assert(r.size() == 3);
static_assert(r[0] == ^C);
```

§ 3.1.4 `template_parameters_of`

```
vector<info> template_parameters_of(info);
```

Given the reflection of a template, `template_parameters_of` returns an array of token sequences containing the template's parameter names (a parameter pack counts as one parameter and is followed by `...`). To avoid name clashes, the implementation chooses unique names. (Our examples use `"_T"` followed by a number.) For function templates, it is guaranteed that `declaration_of` and `template_parameters_of` will use the same names. For example, `template_parameters_of(^A)` may yield the token sequences `^{_T0}`, `^{_T1}`, and `^{_T2...}`.

For explicit specializations of class templates, `template_parameters_of` returns an empty vector:

```
template <typename T> class A;
template <> class A<int>;

// r refers to A<int>
constexpr auto r = template_alternatives_of(^A)[1];    // pick
                specialization
static_assert(template_parameters_of(r).empty());
```

For partial specializations of class templates, `template_parameters_of` returns the parameters of the partial specialization (not those of the primary template). For example:

```
template <typename T> class A;
template <typename T, typename U> class A<std::tuple<T, U>>;
constexpr auto r = template_alternatives_of(^A)[1];    // pick
                specialization
static_assert(template_parameters_of(r).size() == 2);
```

Given a class template `A` that has explicit instantiations and/or partial specializations declared, referring the template by name in the call `template_parameters_of(^A)` is not ambiguous—it refers to the primary declaration of the template.

Calling `template_parameters_of` against a non-template entity, as in `template_parameters_of(^int)` or `template_parameters_of(^std::vector<int>)`, fails to evaluate to a constant expression.

§ 3.1.5 `template_parameter_list`

```
constexpr auto template_parameter_list(info) -> info;
```

Returns the result of `template_parameters_of` as a comma-separated list in a token sequence. Variadics, if any, are followed by `....`.

§ 3.1.6 `attributes_of`, `add_attribute`, `remove_attribute`

```
vector<info> attributes_of(info);
info add_attribute(info, info);
info remove_attribute(info, info);
```

`attributes_of` returns all attributes associated with `info`. The intent is to extend the functionality of `attributes_of` as defined in [P3385R0] to template declarations.

`add_attribute(r, attr)` returns a reflection that adds the attribute `attr` to the given reflection `r`. If `r` already had the attribute, it is returned.

`remove_attribute` returns a reflection that removes the attribute `attr` from the given reflection `r`. If `r` did not have the attribute, it is returned.

We defer details and particulars to [P3385R0].

§ 3.1.7 **set_requires_clause, add_requires_clause_conjunction, add_requires_clause_disjunction**

```
info set_requires_clause(info, info);
info add_requires_clause_conjunction(info, info);
info add_requires_clause_disjunction(info, info);
```

Given the reflection of a template, `set_requires_clause` returns a reflection of an identical template but with the `requires` clause replaced by the token sequence given in the second parameter. The existing `requires` clause in the template, if any, is ignored. Example:

```
template <typename T>
requires (sizeof(T) > 64)
void fun(const T& value) { ... }

consteval {
    auto f = ^fun;
    queue_injection(^{
        [:\(set_requires_clause(set_name(declaration_of(f), "my_fun"),
            ^{ sizeof(\tokens(template_parameters_of(f)[0])) == 128 }):];
    });
}

// Equivalent code:
// template <typename T>
// requires (sizeof(T) == 128)
// void my_fun(const T& value);
```

The functions `add_requires_clause_conjunction` and `add_requires_clause_disjunction` compose the given token sequence with the existing `requires` clause. (If no such clause exists, it is assumed to be the expression `true`.) The `add_requires_clause_conjunction` metafunction performs a conjunction (logical “and”) between the clause given in the token sequence and the existing token sequence. The

`add_requires_clause_disjunction` metafunction performs a disjunction (logical “or”) between the clause given in the token sequence and the existing token sequence.

Calling `set_requires_clause`, `add_requires_clause_conjunction`, or `add_requires_clause_disjunction` against a non-template, as in `set_requires_clause(^int, ^{true})`, fails to evaluate to a constant expression.

§ 3.1.8 `is_primary_template`

```
bool is_primary_template(info);
```

Returns `true` if `info` refers to the primary declaration of a class template or variable template, `false` in all other cases (including cases in which the query does not apply, such as `is_primary_template(^int)`). In particular, this metafunction is useful for distinguishing between a primary template and its explicit specializations or partial specializations. For example:

```
template <class> class A;
template <> class A<int>;
static_assert(is_primary_template(^A));
static_assert(is_primary_template(template_alternatives_of(^A)[0]));
static_assert(!is_primary_template(template_alternatives_of(^A)[1]));
static_assert(!is_primary_template(^int));
```

§ 3.1.9 `overloads_of`

```
vector<info> overloads_of(info);
```

Returns an array of all overloads of a given function, usually passed in as a reflection of an identifier (e.g., `^func`). All overloads are included, template and nontemplate. To distinguish functions from function templates, `is_function_template` ([P2996R7]) can be used. Example:

```
void f();
template <typename T> void f(const T&);

// Name f has two overloads present.
static_assert(overloads_of(^f).size() == 2);
```

The order of overloads in the results is the same as encountered in source code.

If a function `f` has only one declaration (no overloads), `overloads_of(^f)` returns a vector with one element, which is equal to `^f`. Therefore, for such functions, `^f` and `overloads_of(^f)[0]` can be used interchangeably.

Each info returned by `overloads_of` is the reflection of a fully resolved symbol that doesn't participate in any further overload resolution. In other words, for any function `f` (overloaded or not), `overloads_of(overloads_of(f)[i])` is the same as `overloads_of(f)[i]` for all appropriate values of `i`.

§ 3.1.10 `is_inline`

```
bool is_inline(info);
```

Returns `true` if and only if the given reflection handle refers to a function, function template, or namespace that has the `inline` specifier. In all other cases, such as `is_inline(int)`, returns false.

Note that this metafunction has applicability beyond templates as it applies to regular functions and namespaces. Because of this it is possible to migrate this metafunction to a future revision of P2996.

§ 3.1.11 `is_const`, `is_explicit`, `is_volatile`, `is_rvalue_reference_qualified`, `is_lvalue_reference_qualified`, `is_static_member`, `has_static_linkage`, `is_noexcept`

```
bool is_const(info);
bool is_explicit(info);
bool is_volatile(info);
bool is_rvalue_reference_qualified(info);
bool is_lvalue_reference_qualified(info);
bool is_static_member(info);
bool has_static_linkage(info);
bool is_noexcept(info);
```

Extends the homonym metafunctions defined by [\[P2996R7\]](#) to function templates.

For `is_noexcept` and `is_explicit`, `true` is returned if the `noexcept` and `explicit` clause, respectively, is predicated.

§ 3.1.12 `set_noexcept_clause`, `add_noexcept_clause_conjunction`, `add_noexcept_clause_disjunction`

```
info set_noexcept_clause(info, info);
info add_noexcept_clause_conjunction(info, info);
info add_noexcept_clause_disjunction(info, info);
```

Given the reflection of a template, `set_noexcept_clause` returns a reflection of an identical template but with the `noexcept` clause replaced by the token sequence given in the second parameter. The existing `noexcept` clause in the template, if any, is ignored. Example:

```

template <typename T>
noexcept (sizeof(T) > 64)
void fun(const T& value) { ... }

consteval {
    auto f = ^^fun;
    queue_injection(^^{
        [:\(set_noexcept_clause(set_name(declaration_of(f), "my_fun")),
            ^^{ sizeof(\tokens(template_parameters_of(f)[0])) == 128 }):];
    });
}

// Equivalent code:
// template <typename T>
// noexcept (sizeof(T) == 128)
// void my_fun(const T& value);

```

The functions `add_noexcept_clause_conjunction` and `add_noexcept_clause_disjunction` compose the given token sequence with the existing `noexcept` clause. (If no such clause exists, it is assumed to be the expression `true`.) The `add_noexcept_clause_conjunction` metafunction performs a conjunction (logical “and”) between the clause given in the token sequence and the existing token sequence. The `add_noexcept_clause_disjunction` metafunction performs a disjunction (logical “or”) between the clause given in the token sequence and the existing token sequence.

Calling `set_noexcept_clause`, `add_noexcept_clause_conjunction`, or `add_noexcept_clause_disjunction` against a non-template, as in `set_noexcept_clause(^^int, ^^{true})`, fails to evaluate to a constant expression.

§ 3.1.13 `set_explicit_clause`, `add_explicit_clause_conjunction`, `add_explicit_clause_disjunction`

```

info set_explicit_clause(info, info);
info add_explicit_clause_conjunction(info, info);
info add_explicit_clause_disjunction(info, info);

```

Given the reflection of a template, `set_explicit_clause` returns a reflection of an identical template but with the `explicit` clause replaced by the token sequence given in the second parameter. The existing `explicit` specifier in the template, if any, is ignored.

The functions `add_explicit_clause_conjunction` and `add_explicit_clause_disjunction` compose the given token sequence with the existing `explicit` clause. (If no such clause exists, it is assumed to be the expression `true`.) The `add_explicit_clause_conjunction` metafunction performs a conjunction (logical “and”) between the clause given in the token sequence and the existing token sequence. The `add_explicit_clause_disjunction` metafunction performs a disjunction (logical “or”) between the clause given in the token sequence and the existing token sequence.

Calling `set_explicit_clause`, `add_explicit_clause_conjunction`, or `add_explicit_clause_disjunction` against a non-template, as in `set_explicit_clause(^int, ^{true})`, fails to evaluate to a constant expression.

§ 3.1.14 `is_declared_constexpr`, `is_declared_consteval`

```
bool is_declared_constexpr(info);  
bool is_declared_consteval(info);
```

Returns `true` if and only if `info` refers to a declaration that is `constexpr` or `consteval`, respectively. In all other cases, returns `false`.

§ 3.1.15 `parameters_of`

```
vector<info> parameters_of(info);
```

Returns an array of token sequences containing the parameter names of the function template represented by `info`. Parameter names are chosen by the implementation. It is guaranteed that the names returned by `parameters_of(f)` are consistent with the names returned by `parameters_of(f)` for any reflection `f` of a function template.

This function bears similarities with `parameters_of` in [P3096R3]. However, one key difference is that `parameters_of` returns reflection of types, whereas `template_return_type_of` returns the token sequence of the return type (given that the type is often not known before instantiation or may be `auto` etc).

§ 3.1.16 `parameter_list`

```
info parameter_list(info);
```

Convenience function that returns the parameters returned by `parameters_of` in the form of a comma-separated list. The result is returned as a token sequence.

§ 3.1.17 `forward_parameter_list`

```
info forward_parameter_list(info);
```

Convenience function that returns the parameters returned by `parameters_of` in the form of a comma-separated list, each passed to `std::forward` appropriately. The result is returned as a token sequence and can be spliced in a call to forward all parameters to another function.

§ 3.1.18 `set_trailing_requires_clause`, `add_trailing_requires_clause_conjunction`

add_trailing_requires_clause_disjunction

```
info set_trailing_requires_clause(info, info);
info add_trailing_requires_clause_conjunction(info, info);
info add_trailing_requires_clause_disjunction(info, info);
```

Given the reflection of a template, `set_trailing_requires_clause` returns a reflection of an identical template but with the `requires` clause replaced by the token sequence given in the second parameter. The existing `requires` clause in the template, if any, is ignored. Example:

```
template <typename T>
requires (sizeof(T) > 64)
void fun(const T& value) { ... }

consteval {
    auto f = ^^fun;
    queue_injection(^^{
        [:\(set_trailing_requires_clause(set_name(declaration_of(f),
            "my_fun")),
        ^^{ sizeof(\tokens(template_parameters_of(f)[0])) == 128 }):];
    });
}

// Equivalent code:
// template <typename T>
// requires (sizeof(T) == 128)
// void my_fun(const T& value);
```

The functions `add_trailing_requires_clause_conjunction` and `add_trailing_requires_clause_disjunction` compose the given token sequence with the existing `requires` clause. (If no such clause exists, it is assumed to be the expression `true`.) The `add_trailing_requires_clause_conjunction` metafunction performs a conjunction (logical “and”) between the clause given in the token sequence and the existing token sequence. The `add_trailing_requires_clause_disjunction` metafunction performs a disjunction (logical “or”) between the clause given in the token sequence and the existing token sequence.

Calling `set_trailing_requires_clause`, `add_trailing_requires_clause_conjunction`, or `add_trailing_requires_clause_disjunction` against a non-template, as in `set_trailing_requires_clause(^^int, ^^{true})`, fails to evaluate to a constant expression.

§ 4 Metafunctions for Iterating Members of Class Templates

The metafunctions described above need to be complemented with metafunctions that enumerate the contents of a class template. Fortunately, [\[P2996R7\]](#) already defines number of metafunctions for doing so: `members_of`, `static_data_members_of`, `nonstatic_data_members_of`,

`nonstatic_data_members_of`, `enumerators_of`, `subobjects_of`. To these, P2996 adds access-aware metafunctions `accessible_members_of`, `accessible_bases_of`, `accessible_static_data_members_of`, `accessible_nonstatic_data_members_of`, and `accessible_subobjects_of`. Here, we propose that these functions are extended in semantics to also support info representing class templates. The reflections of members of class templates, however, are not the same as the reflection of corresponding members of regular classes;

One important addendum to P2996 that is crucial for code generation is that when applied to members, these discovery metafunctions return members in lexical order. Otherwise, attempts to implement identity will fail for class templates. Consider:

```
template <typename T>
struct Container {
    using size_type = size_t;
    size_type size() const;
}
```

§ 5 Future Work

Like [P2996R7], this proposal is designed to grow by accretion. Future revisions will add more primitives that complement the existing ones and improve the comprehensiveness of the reflection facility for templates.

This proposal is designed to interoperate with [P2996R7], [P3157R1], [P3294R2], and [P3385R0]. Changes in these proposals may influence this proposal. Also, certain metafunctions proposed herein (such as `is_inline`) may migrate to these proposals.

§ 6 References

[P2996R7] Wyatt Childers, Peter Dimov, Dan Katz, Barry Revzin, Andrew Sutton, Faisal Vali, and Daveed Vandevoorde. 2024-10-12. Reflection for C++26.

<https://wg21.link/p2996r7>

[P3096R3] Adam Lach, Walter Genovese. 2024-09-15. Function Parameter Reflection in Reflection for C++26.

<https://wg21.link/p3096r3>

[P3157R1] Andrei Alexandrescu, Barry Revzin, Bryce Lebach, Michael Garland. 2024-05-22. Generative Extensions for Reflection.

<https://wg21.link/p3157r1>

[P3294R1] Barry Revzin, Andrei Alexandrescu, Daveed Vandevoorde. 2024-07-16. Code Injection with Token Sequences.

<https://wg21.link/p3294r1>

[P3294R2] Andrei Alexandrescu, Barry Revzin, and Daveed Vandevoorde. 2024-10-15. Code Injection with Token Sequences.

<https://wg21.link/p3294r2>

[P3385R0] Aurelien Cassagnes, Aurelien Cassagnes, Roman Khoroshikh, Anders Johansson. 2024-09-16.
Attributes reflection.

<https://wg21.link/p3385r0>

[P3420R0] Andrei Alexandrescu, Barry Revzin, Daveed Vandevoorde, Michael Garland. 2024-10-16.
Reflection of Templates.

<https://wg21.link/p3420r0>